

TUGAS M4 - DATA MINING

CLASSIFICATION (K-NN)

GUSTHI PANGESTU

D4 IT D | 3124600098



```
Tugas 3 > jawaban > 01_load_dataset.py > base_dir
1  import os
2  import pandas as pd
3
4  base_dir = os.path.dirname(os.path.abspath(__file__))
5  csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
6  if not os.path.exists(csv_path):
7      csv_path = 'data/titanic.csv'
8
9  dataset = pd.read_csv(csv_path)
10
11  print("=== 1. Load Dataset Train (titanic.csv) ===")
12  print(f"File Path : {csv_path}")
13  print(f"Dimensi   : {dataset.shape[0]} baris, {dataset.shape[1]} kolom\n")
14
15  print("--- Sampel 5 Baris Data Acak (Random Sample) ---")
16  print(dataset.sample(5, random_state=42))
17
18  print("\n--- Ringkasan Kolom ---")
19  print(dataset.info())
```

1. LOAD DATASET

Load Dataset ***titanic.csv*** terdapat 891 baris dan 12 kolom menggunakan Pandas, lalu mengeksplorasi sampel acak 5 baris data serta struktur kolomnya di mana saya menemukan adanya nilai kosong (missing values) pada fitur Age, Cabin, dan Embarked yang perlu dibersihkan pada tahap selanjutnya.



HASIL LOAD DATASET

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban> python 01_load_dataset.py
=== 1. Load Dataset Train (titanic.csv) ===
File Path : C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban\..\data\titanic.csv
Dimensi   : 891 baris, 12 kolom

--- Sampel 5 Baris Data Acak (Random Sample) ---

```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
709	710	1	3	Moubarek, Master. Halim Gonios ("William George")	male	NaN	1	1	2661	15.2458	NaN	C
439	440	0	2	Kvillner, Mr. Johan Henrik Johannesson	male	31.0	0	0	C.A. 18723	10.5000	NaN	S
840	841	0	3	Alhomaki, Mr. Ilmari Rudolf	male	20.0	0	0	SOTON/O2 3101287	7.9250	NaN	S
720	721	1	2	Harper, Miss. Annie Jessie "Nina"	female	6.0	0	1	248727	33.0000	NaN	S
39	40	1	3	Nicola-Yarred, Miss. Jamila	female	14.0	1	0	2651	11.2417	NaN	C

```

--- Ringkasan Kolom ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```



Tugas 3 > jawaban > 02_load_test_dataset.py > base_dir

```
1 import os
2 import pandas as pd
3
4 base_dir = os.path.dirname(os.path.abspath(__file__))
5 csv_path = os.path.join(base_dir, '..', 'data', 'titanic_test.csv')
6 if not os.path.exists(csv_path):
7     csv_path = 'data/titanic_test.csv'
8
9 test_dataset = pd.read_csv(csv_path)
10
11 print("=== 2. Load Dataset Test (titanic_test.csv) ===")
12 print(f"File Path : {csv_path}")
13 print(f"Dimensi   : {test_dataset.shape[0]} baris, {test_dataset.shape[1]} kolom\n")
14
15 print("--- Sampel 5 Baris Data Acak (Random Sample) ---")
16 print(test_dataset.sample(5, random_state=42))
17
18 print("\n--- Ringkasan Kolom ---")
19 print(test_dataset.info())
```

2. LOAD DATASET

Load Dataset *titanic_test.csv* menggunakan Pandas untuk memeriksa kesiapan data evaluasi, mengecek dimensinya, mengambil 5 sampel acak (*random_state=42*), serta meninjau struktur kolomnya lewat *.info()* guna memastikan konsistensi format fitur dengan data latih sebelumnya.



HASIL LOAD DATASET

```
None
● PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban> python 02_load_test_dataset.py
=== 2. Load Dataset Test (titanic_test.csv) ===
File Path : C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban\..\data\titanic_test.csv
Dimensi   : 418 baris, 11 kolom

--- Sampel 5 Baris Data Acak (Random Sample) ---

```

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
321	1213	3	Krekorian, Mr. Neshan	male	25.0	0	0	2654	7.2292	F E57	C
324	1216	1	Kreuchen, Miss. Emilie	female	39.0	0	0	24160	211.3375	NaN	S
388	1280	3	Canavan, Mr. Patrick	male	21.0	0	0	364858	7.7500	NaN	Q
56	948	3	Cor, Mr. Bartol	male	35.0	0	0	349230	7.8958	NaN	S
153	1045	3	Klasen, Mrs. (Hulda Kristina Eugenia Lofqvist)	female	36.0	0	2	350405	12.1833	NaN	S

```

--- Ringkasan Kolom ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  418 non-null    int64
1   Pclass       418 non-null    int64
2   Name         418 non-null    object
3   Sex          418 non-null    object
4   Age          332 non-null    float64
5   SibSp        418 non-null    int64
6   Parch        418 non-null    int64
7   Ticket       418 non-null    object
8   Fare         417 non-null    float64
9   Cabin        91 non-null     object
10  Embarked     418 non-null    object
dtypes: float64(2), int64(4), object(5)
memory usage: 36.1+ KB
None
○ PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban> 
```



3. EKSTRAKSI FITUR & CLEANING DATA

```
Tugas 3 > jawaban > 03_train_data_fitur.py > ...
1  import os
2  import pandas as pd
3
4  base_dir = os.path.dirname(os.path.abspath(__file__))
5  csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
6  if not os.path.exists(csv_path):
7      csv_path = 'data/titanic.csv'
8
9  dataset = pd.read_csv(csv_path)
10
11 # Ambil fitur (Age, Fare)
12 train_features = dataset[["Age", "Fare"]]
13
14 # Catat posisi baris yang memiliki missing value
15 pos_missing_train = train_features.isna().any(axis=1)
16
17 # Hapus baris yang memiliki missing value
18 train_data = train_features[~pos_missing_train].copy()
19
20 print("=== 3. Fitur Train Data (Age, Fare) & Pembersihan Missing Value ===")
21 print(f"Jumlah baris awal          : {len(train_features)}")
22 print(f"Jumlah baris missing value    : {pos_missing_train.sum()}")
23 print(f"Jumlah baris bersih (train_data): {len(train_data)}\n")
24
25 print("--- Sampel Indeks Baris yang Memiliki Missing Value (Acak) ---")
26 sample_missing_idx = train_features[pos_missing_train].sample(10, random_state=42).index.tolist()
27 print(sample_missing_idx)
28
29 print("\n--- Sampel 5 Baris train_data Secara Acak (Random Sample) ---")
30 print(train_data.sample(5, random_state=42))
```

```
● PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban> python 0
=== 3. Fitur Train Data (Age, Fare) & Pembersihan Missing Value ===
Jumlah baris awal          : 891
Jumlah baris missing value    : 177
Jumlah baris bersih (train_data): 714

--- Sampel Indeks Baris yang Memiliki Missing Value (Acak) ---
[82, 235, 667, 158, 347, 65, 573, 863, 531, 669]

--- Sampel 5 Baris train_data Secara Acak (Random Sample) ---
      Age  Fare
149  42.0  13.00
407   3.0  18.75
53   29.0  26.00
369  24.0  69.30
818  43.0   6.45
○ PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
```

Mengekstrak subset fitur numerik (**Age & Fare**) dari dataset awal, lalu mendeteksi dan menghapus 177 baris yang memiliki missing value sehingga menghasilkan 714 baris data bersih yang siap diolah ke tahap normalisasi.

4. SELEKSI FITUR DATA UJI

```
Tugas 3 > jawaban > 04_test_data_fitur.py > ...
1 import os
2 import pandas as pd
3
4 base_dir = os.path.dirname(os.path.abspath(__file__))
5 csv_path = os.path.join(base_dir, '..', 'data', 'titanic_test.csv')
6 if not os.path.exists(csv_path):
7     csv_path = 'data/titanic_test.csv'
8
9 test_dataset = pd.read_csv(csv_path)
10
11 test_features = test_dataset[["Age", "Fare"]]
12
13 pos_missing_test = test_features.isna().any(axis=1)
14
15 test_data = test_features[~pos_missing_test].copy()
16
17 print("=== 4. Fitur Test Data (Age, Fare) & Pembersihan Missing Value ===")
18 print(f"Jumlah baris awal          : {len(test_features)}")
19 print(f"Jumlah baris missing value    : {pos_missing_test.sum()}")
20 print(f"Jumlah baris bersih (test_data): {len(test_data)}\n")
21
22 print("--- Sampel Indeks Baris yang Memiliki Missing Value (Acak) ---")
23 sample_missing_idx = test_features[pos_missing_test].sample(10, random_state=42).index.tolist()
24 print(sample_missing_idx)
25
26 print("\n--- Sampel 5 Baris test_data Secara Acak (Random Sample) ---")
27 print(test_data.sample(5, random_state=42))
```

```
● PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban> python
=== 4. Fitur Test Data (Age, Fare) & Pembersihan Missing Value ===
Jumlah baris awal          : 418
Jumlah baris missing value    : 87
Jumlah baris bersih (test_data): 331

--- Sampel Indeks Baris yang Memiliki Missing Value (Acak) ---
[358, 10, 132, 116, 83, 297, 65, 102, 36, 301]

--- Sampel 5 Baris test_data Secara Acak (Random Sample) ---
      Age      Fare
27  22.5    7.2250
389   6.0   21.0750
89    2.0   23.0000
280  23.0    8.6625
68   31.0   28.5375
○ PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
```

Mengekstrak subset fitur numerik (**Age & Fare**) dari dataset uji (**titanic_test.csv**), lalu mendeteksi dan menghapus 87 baris yang memiliki missing value sehingga menghasilkan 331 baris data bersih yang siap dievaluasi.

5. EKSTRAKSI KELAS TARGET

```
Tugas 3 > jawaban > 05_train_label.py > ...
1 import os
2 import pandas as pd
3
4 base_dir = os.path.dirname(os.path.abspath(__file__))
5 csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
6 if not os.path.exists(csv_path):
7     csv_path = 'data/titanic.csv'
8
9 dataset = pd.read_csv(csv_path)
10
11 # Identifikasi baris missing value pada fitur (Age, Fare)
12 train_features = dataset[["Age", "Fare"]]
13 pos_missing_train = train_features.isna().any(axis=1)
14
15 # Ambil label target Survived hanya untuk baris yang bersih
16 train_label = dataset.loc[~pos_missing_train, "Survived"]
17
18 print("=== 5. Label Train (Survived) ===")
19 print(f"Jumlah label train (valid) : {len(train_label)}")
20 print(f"\nDistribusi Kelas:\n{train_label.value_counts()}")
21
22 print("\n--- Sampel 10 Label Secara Acak (Random Sample) ---")
23 print(train_label.sample(10, random_state=42))
```

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\ja
=== 5. Label Train (Survived) ===
Jumlah label train (valid) : 714

Distribusi Kelas:
Survived
0      424
1      290
Name: count, dtype: int64

--- Sampel 10 Label Secara Acak (Random Sample) ---
149      0
407      1
53       1
369      1
818      0
549      1
85       1
774      1
100      0
816      0
Name: Survived, dtype: int64
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\ja
```

Mengekstrak kolom target **Survived** khusus untuk 714 baris data yang sudah bersih dari missing value, menghasilkan distribusi kelas biner yaitu 424 data label 0 (tidak selamat) dan 290 data label 1 (selamat) sebagai acuan label latih model.

6. LABEL DATA UJI (TEST LABEL)

```
Tugas 3 > jawaban > 06_test_label.py > ...
1 import os
2 import pandas as pd
3
4 base_dir = os.path.dirname(os.path.abspath(__file__))
5 data_dir = os.path.join(base_dir, '..', 'data')
6
7 test_path = os.path.join(data_dir, 'titanic_test.csv')
8 if not os.path.exists(test_path):
9     test_path = 'data/titanic_test.csv'
10
11 testlabel_path = os.path.join(data_dir, 'titanic_testlabel.csv')
12 if not os.path.exists(testlabel_path):
13     testlabel_path = 'data/titanic_testlabel.csv'
14
15 # Identifikasi baris missing value pada test data (Age, Fare)
16 test_dataset = pd.read_csv(test_path)
17 test_features = test_dataset[["Age", "Fare"]]
18 pos_missing_test = test_features.isna().any(axis=1)
19
20 # Baca label test
21 raw_test_label = pd.read_csv(testlabel_path)
22 target_col = "Survived" if "Survived" in raw_test_label.columns else raw_test_label.columns[0]
23 test_label = raw_test_label.loc[~pos_missing_test, target_col]
24
25 print("=== 6. Label Test dari titanic_testlabel.csv ===")
26 print(f"File Label Test      : {testlabel_path}")
27 print(f"Kolom Target Digunakan  : '{target_col}'")
28 print(f"Jumlah baris raw label   : {len(raw_test_label)}")
29 print(f"Jumlah test_label valid  : {len(test_label)}")
30 print(f"\nDistribusi Kelas Test:\n{test_label.value_counts()}")
31
32 print("\n--- Sampel 10 Label Secara Acak (Random Sample) ---")
33 print(test_label.sample(10, random_state=42))
```

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\ja
=== 6. Label Test dari titanic_testlabel.csv ===
File Label Test      : C:\Users\Gusthi Pangestu\
Kolom Target Digunakan  : 'Survived'
Jumlah baris raw label   : 418
Jumlah test_label valid  : 331

Distribusi Kelas Test:
Survived
0      204
1      127
Name: count, dtype: int64

--- Sampel 10 Label Secara Acak (Random Sample) ---
27      0
389     0
89      0
280     1
68      0
207     0
354     1
257     0
338     0
245     0
Name: Survived, dtype: int64
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\ja
```

Mengekstrak label target **Survived** dari file **titanic_testlabel.csv** khusus untuk 331 baris data uji yang bersih dari missing value, menghasilkan distribusi 204 data label 0 (tidak selamat) dan 127 data label 1 (selamat) sebagai *ground truth* evaluasi model.

7. MIN - MAX (0 - 1)

```
Tugas 3 > jawaban > 07_normalisasi_train_minmax.py > ...
1 import os
2 import pandas as pd
3
4 base_dir = os.path.dirname(os.path.abspath(__file__))
5 csv_path = os.path.join(base_dir, '..', 'data', 'titanic.csv')
6 if not os.path.exists(csv_path):
7     csv_path = 'data/titanic.csv'
8
9 dataset = pd.read_csv(csv_path)
10
11 # Ekstraksi fitur train & hapus missing value
12 train_features = dataset[["Age", "Fare"]]
13 pos_missing_train = train_features.isna().any(axis=1)
14 train_data = train_features[~pos_missing_train].copy()
15
16 # 7. Normalisasi Min-Max (0 - 1) & catat min dan max
17 min_val = train_data.min()
18 max_val = train_data.max()
19
20 norm_train_data = (train_data - min_val) / (max_val - min_val)
21
22 print("=== 7. Normalisasi Min-Max 0-1 pada train_data ===")
23 print("Nilai Minimum Atribut Train:")
24 print(min_val)
25 print("\nNilai Maksimum Atribut Train:")
26 print(max_val)
27
28 # Mengambil sampel acak pakai index yang sama untuk membandingkan sebelum vs sesudah
29 sample_indices = train_data.sample(5, random_state=42).index
30
31 print("\n--- Sampel 5 Baris train_data Sebelum Normalisasi (Acak) ---")
32 print(train_data.loc[sample_indices])
33
34 print("\n--- Sampel 5 Baris train_data Setelah Normalisasi (Acak) ---")
35 print(norm_train_data.loc[sample_indices])
36
```

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
=== 7. Normalisasi Min-Max 0-1 pada train_data ===
Nilai Minimum Atribut Train:
Age      0.42
Fare      0.00
dtype: float64

Nilai Maksimum Atribut Train:
Age      80.0000
Fare     512.3292
dtype: float64

--- Sampel 5 Baris train_data Sebelum Normalisasi (Acak) ---
      Age  Fare
149  42.0  13.00
407   3.0  18.75
53   29.0  26.00
369  24.0  69.30
818  43.0   6.45

--- Sampel 5 Baris train_data Setelah Normalisasi (Acak) ---
      Age      Fare
149  0.522493  0.025374
407  0.032420  0.036598
53   0.359135  0.050749
369  0.296306  0.135265
818  0.535059  0.012590
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
```

Normalisasi Min-Max pada fitur **Age** & **Fare** data latih dengan menghitung nilai minimum dan maksimumnya, lalu memetakan seluruh nilai ke rentang skala 0 hingga 1 agar perbedaan skala antarfitur tidak mendominasi model klasifikasi.



8. NORMALISASI MIN - MAX DATA TEST



```
Tugas 3 > jawaban > 08_normalisasi_test_minmax.py > ...
1  import os
2  import pandas as pd
3
4  base_dir = os.path.dirname(os.path.abspath(__file__))
5  data_dir = os.path.join(base_dir, '..', 'data')
6
7  train_path = os.path.join(data_dir, 'titanic.csv')
8  test_path = os.path.join(data_dir, 'titanic_test.csv')
9
10 if not os.path.exists(train_path):
11     train_path = 'data/titanic.csv'
12 if not os.path.exists(test_path):
13     test_path = 'data/titanic_test.csv'
14
15 # Train_data diperoleh min & max (Langkah 7)
16 dataset = pd.read_csv(train_path)
17 train_features = dataset[["Age", "Fare"]]
18 pos_missing_train = train_features.isna().any(axis=1)
19 train_data = train_features[~pos_missing_train].copy()
20
21 min_val = train_data.min()
22 max_val = train_data.max()
23
24 # Preprocessing test_data
25 test_dataset = pd.read_csv(test_path)
26 test_features = test_dataset[["Age", "Fare"]]
27 pos_missing_test = test_features.isna().any(axis=1)
28 test_data = test_features[~pos_missing_test].copy()
29
```

```
30 # 8. Normalisasi Min-Max test_data menggunakan min & max Langkah 7 (dari train_data)
31 norm_test_data = (test_data - min_val) / (max_val - min_val)
32
33 print("=== 8. Normalisasi Min-Max 0-1 pada test_data (Menggunakan Min & Max Train) ===")
34 print("Nilai Min Acuan (dari Train):")
35 print(min_val)
36 print("\nNilai Max Acuan (dari Train):")
37 print(max_val)
38
39 # Ambil sampel acak menggunakan index yang sama untuk membandingkan sebelum vs sesudah
40 sample_indices = test_data.sample(5, random_state=42).index
41
42 print("\n--- Sampel 5 Baris test_data Sebelum Normalisasi (Acak) ---")
43 print(test_data.loc[sample_indices])
44
45 print("\n--- Sampel 5 Baris test_data Setelah Normalisasi (Acak) ---")
46 print(norm_test_data.loc[sample_indices])
```

Normalisasi Min-Max pada fitur **Age** & **Fare** data uji (**test_data**) menggunakan acuan nilai minimum dan maksimum dari data latih (**train_data**), guna menyetarakan skala data evaluasi tanpa menyebabkan kebocoran data (**data leakage**).



HASIL

NORMALISASI MIN - MAX DATA TEST

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
=== 8. Normalisasi Min-Max 0-1 pada test_data (Menggunakan
Nilai Min Acuan (dari Train):
Age      0.42
Fare      0.00
dtype: float64

Nilai Max Acuan (dari Train):
Age      80.0000
Fare     512.3292
dtype: float64

--- Sampel 5 Baris test_data Sebelum Normalisasi (Acak) ---
      Age      Fare
27    22.5    7.2250
389    6.0   21.0750
89     2.0   23.0000
280   23.0    8.6625
68    31.0   28.5375

--- Sampel 5 Baris test_data Setelah Normalisasi (Acak) ---
      Age      Fare
27  0.277457  0.014102
389  0.070118  0.041136
89   0.019854  0.044893
280  0.283740  0.016908
68   0.384267  0.055701
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
```



9. PREDIKSI DATA UJI DENGAN K-NN



```
Tugas 3 > jawaban > 09_knn_classification.py > ...
1  import os
2  import pandas as pd
3  from sklearn.neighbors import KNeighborsClassifier
4
5  base_dir = os.path.dirname(os.path.abspath(__file__))
6  data_dir = os.path.join(base_dir, '..', 'data')
7
8  train_path = os.path.join(data_dir, 'titanic.csv')
9  test_path = os.path.join(data_dir, 'titanic_test.csv')
10
11 if not os.path.exists(train_path):
12     train_path = 'data/titanic.csv'
13 if not os.path.exists(test_path):
14     test_path = 'data/titanic_test.csv'
15
16 # Preprocessing Train Data
17 dataset = pd.read_csv(train_path)
18 train_features = dataset[["Age", "Fare"]]
19 pos_missing_train = train_features.isna().any(axis=1)
20 train_data = train_features[~pos_missing_train].copy()
21 train_label = dataset.loc[~pos_missing_train, "Survived"]
22
23 min_val = train_data.min()
24 max_val = train_data.max()
25 train_data = (train_data - min_val) / (max_val - min_val)
26
27 # Preprocessing Test Data
28 test_dataset = pd.read_csv(test_path)
29 test_features = test_dataset[["Age", "Fare"]]
30 pos_missing_test = test_features.isna().any(axis=1)
31 test_data = test_features[~pos_missing_test].copy()
32 test_data = (test_data - min_val) / (max_val - min_val)
33
34 print("=== 9. Klasifikasi k-NN (k = 1 s.d. 15, weights='distance') ===")
35 print(f>Data Pelatihan (Train) : {len(train_data)} sampel")
36 print(f>Data Pengujian (Test) : {len(test_data)} sampel\n")
37
38 print(f'{k':<5} | {'Prediksi 0 (Tidak Selamat)':<27} | {'Prediksi 1 (Selamat)':<20}")
39 print("-" * 57)
```

```
40
41 for k in range(1, 16):
42     # Inisialisasi model k-NN dengan bobot distance
43     knn = KNeighborsClassifier(n_neighbors=k, weights="distance")
44
45     # Fit data pelatihan
46     knn.fit(train_data, train_label)
47
48     # Prediksi hasil kelas
49     class_result = knn.predict(test_data)
50
51     count_0 = (class_result == 0).sum()
52     count_1 = (class_result == 1).sum()
53     print(f'{k:<5} | {count_0:<27} | {count_1:<20}")
```

Melatih model klasifikasi k-Nearest Neighbors (k-NN) berbobot jarak (***weights='distance'***) pada data latih hasil normalisasi Min-Max, lalu memprediksi kelas target data uji secara berulang untuk nilai ***k*** = 1 hingga 15 guna melihat perbandingan jumlah prediksi label 0 dan 1.



HASIL

PREDIKSI DATA UJI DENGAN K-NN

```
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
=== 9. Klasifikasi k-NN (k = 1 s.d. 15, weights='distance')
Data Pelatihan (Train) : 714 sampel
Data Pengujian (Test)  : 331 sampel

k      | Prediksi 0 (Tidak Selamat) | Prediksi 1 (Selamat)
-----|-----|-----
1      | 191                         | 140
2      | 195                         | 136
3      | 185                         | 146
4      | 194                         | 137
5      | 192                         | 139
6      | 193                         | 138
7      | 193                         | 138
8      | 191                         | 140
9      | 190                         | 141
10     | 193                         | 138
11     | 197                         | 134
12     | 197                         | 134
13     | 196                         | 135
14     | 198                         | 133
15     | 199                         | 132
PS C:\Users\Gusthi Pangestu\Documents\WDM\Tugas 3\jawaban>
```

10. EVALUASI MODEL

```
Tugas 3 > jawaban > 10_precision_error_ratio.py > test_dataset
1 import os
2 import pandas as pd
3 from sklearn.neighbors import KNeighborsClassifier
4
5 base_dir = os.path.dirname(os.path.abspath(__file__))
6 data_dir = os.path.join(base_dir, '..', 'data')
7
8 train_path = os.path.join(data_dir, 'titanic.csv')
9 test_path = os.path.join(data_dir, 'titanic_test.csv')
10 testlabel_path = os.path.join(data_dir, 'titanic_testlabel.csv')
11
12 if not os.path.exists(train_path):
13     train_path = 'data/titanic.csv'
14 if not os.path.exists(test_path):
15     test_path = 'data/titanic_test.csv'
16 if not os.path.exists(testlabel_path):
17     testlabel_path = 'data/titanic_testlabel.csv'
18
19 dataset = pd.read_csv(train_path)
20 train_features = dataset[["Age", "Fare"]]
21 pos_missing_train = train_features.isna().any(axis=1)
22 train_data = train_features[~pos_missing_train].copy()
23 train_label = dataset.loc[~pos_missing_train, "Survived"]
24
25 min_val = train_data.min()
26 max_val = train_data.max()
27 train_data = (train_data - min_val) / (max_val - min_val)
28
29 test_dataset = pd.read_csv(test_path)
30 test_features = test_dataset[["Age", "Fare"]]
31 pos_missing_test = test_features.isna().any(axis=1)
32 test_data = test_features[~pos_missing_test].copy()
33 test_data = (test_data - min_val) / (max_val - min_val)
34
35 # Ambil label pengujian
36 raw_test_label = pd.read_csv(testlabel_path)
37 target_col = "Survived" if "Survived" in raw_test_label.columns else raw_test_label.columns[0]
38 test_label = raw_test_label.loc[~pos_missing_test, target_col]
39
```

```
39
40 print("== 10. Perhitungan Precision Ratio & Error Ratio (k = 1 s.d. 15) ==")
41 print(f"{'k':<5} | {'Precision Ratio':<17} | {'Error Ratio':<15}")
42 print("-" * 43)
43
44 for k in range(1, 16):
45     # Inisialisasi model k-NN dengan weights='distance'
46     knn = KNeighborsClassifier(n_neighbors=k, weights="distance")
47
48     # Fit data pelatihan
49     knn.fit(train_data, train_label)
50
51     # Prediksi hasil kelas
52     class_result = knn.predict(test_data)
53
54     # Menghitung precision ratio (akurasi/skor) dan error ratio
55     precision_ratio = knn.score(test_data, test_label)
56     error_ratio = 1.0 - precision_ratio
57
58     print(f"{'k':<5} | {'precision_ratio':<17.4f} | {'error_ratio':<15.4f}")
```

Mengevaluasi kinerja model k-NN berbobot jarak (***weights='distance'***) pada data uji untuk nilai ***k*** = 1 hingga 15 dengan menghitung *Precision Ratio* (akurasi model) dan *Error Ratio* (***1 - akurasi***) guna menemukan nilai ***k*** paling optimal.



HASIL EVALUASI MODEL

● PS C:\Users\Gusthi Pangestu\Documents\WDM\T

=== 10. Perhitungan Precision Ratio & Error

k	Precision Ratio	Error Ratio
1	0.5498	0.4502
2	0.5619	0.4381
3	0.5438	0.4562
4	0.5408	0.4592
5	0.5347	0.4653
6	0.5438	0.4562
7	0.5498	0.4502
8	0.5378	0.4622
9	0.5408	0.4592
10	0.5438	0.4562
11	0.5559	0.4441
12	0.5619	0.4381
13	0.5529	0.4471
14	0.5650	0.4350
15	0.5619	0.4381

○ PS C:\Users\Gusthi Pangestu\Documents\WDM\T

KESIMPULAN

Rangkaian tahapan praktikum berhasil menangani nilai hilang (***missing values***) pada fitur ***Age*** & ***Fare***, menyetarakan skala data melalui normalisasi Min-Max tanpa *data leakage*, serta mengevaluasi performa model k-NN (***k*** = 1 - 15) menggunakan metrik *Precision Ratio* dan *Error Ratio* secara akurat.

- Integritas Data Terjaga: Penerapan parameter minimum dan maksimum dari data latih ke data uji berhasil mencegah bias serta kebocoran informasi (***data leakage***) selama proses penskalaan fitur.
- Optimasi Parameter Model: Pengujian variasi nilai ***k*** = 1 hingga 15 dengan pembobotan jarak (***distance***) memberikan landasan objektif untuk menentukan konfigurasi terbaik yang meminimalkan tingkat kesalahan (***error ratio***).



TERIMA KASIH

You can access my GitHub repository for the source code :

<https://github.com/GusthiPangestu1906/WDM.git>